

Smart Scheduler Project Pitch

Melodi Parton

CIDM 6325 70

09/13/2025

Abstract

This project pitch introduces the Smart Scheduler, a Django-based web application designed to streamline staff scheduling, overtime tracking, and certification compliance. The system aims to replace fragmented spreadsheets and manual processes with an integrated solution that leverages AI-driven suggestions to reduce conflicts, detect coverage gaps, and enhance compliance management. By implementing modular Django apps, GitHub-based version control, and industry-standard risk mitigation strategies, the project demonstrates a professional-quality minimal viable product deliverable within two weeks. The memo outlines the problems, stakeholders, scope, success metrics, evidence base, and risk register, ensuring a coherent architecture and academically grounded justification.

References:

Layman, M. (2023). *Understand Django: An Exploration of the Django Web Framework*. Self-published.

Knight, D. R. T., Aakre, C. A., Anstine, C. V., Munipalli, B., Biazar, P., Mitri, G., ... Dabrh, A. M. A. (2023). Artificial intelligence for patient scheduling in the real-world health care setting: A metanarrative review. *Health Policy and Technology*, 12(4), Article 100824. <https://doi.org/10.1016/j.hlpt.2023.100824>

Del Gallo, M., Mazzuto, G., Ciarapica, F. E., & Bevilacqua, M. (2023). Artificial intelligence to solve production scheduling problems in real industrial settings: Systematic literature review. *Electronics*, 12(23), 4732. <https://doi.org/10.3390/electronics12234732>

Problem:

Scheduling, vacation requests and training are done with paper requests that are then approved by the manager and handwritten in on a schedule calendar. This creates issues, especially when last-minute requests are placed, or the manager is out of the office. This causes scheduling conflicts, gaps in coverage, and shortages in staff.

Current State

- **Scheduling:** Staff shifts, and availability are tracked across multiple spreadsheets and paper records. There's no centralized system, so updates can be missed or overwritten.
- **Overtime Tracking:** Hours worked beyond regular shifts are inconsistently logged, making it difficult to monitor labor costs, prevent burnout, or comply with overtime policies.
- **Certification Compliance:** Employee training and certifications (e.g., safety, technical, or legal requirements) are monitored manually in scattered files. Expiration dates often slip through the cracks.

Pain Points

1. **Conflicts & Gaps**
 - Double-booking of staff or uncovered shifts.
 - Last-minute scrambles when coverage is discovered missing.
2. **Lapsed Certifications**
 - Employees may be scheduled for duties they're not legally certified for.
 - Risk of non-compliance with industry, safety, or regulatory standards.
3. **Wasted Administrative Time**
 - Managers spend hours reconciling spreadsheets, emails, and paper sign-in sheets.
 - Lack of automation means repetitive manual entry and increased chance of error.
4. **Limited Visibility**
 - Leaders lack a real-time dashboard for staffing, overtime, and compliance.
 - Decisions are reactive rather than proactive, leading to stress and

inefficiency.

Consequences

- **Operational Inefficiency:** Administrative bottlenecks reduce productivity and responsiveness.
- **Financial Impact:** Overtime costs balloon due to poor tracking; scheduling errors may require paying additional staff.
- **Compliance & Safety Risk:** Lapsed certifications expose the organization to legal liability, safety hazards, and reputational harm.
- **Employee Morale:** Constant scheduling mix-ups and compliance issues are causing stress and contribute to dissatisfaction or employee turnover.

Proposed Solution: Centralized Workforce Management App

Core Idea

A single web/mobile app that integrates **scheduling, overtime tracking, and certification compliance** into one platform. It eliminates reliance on spreadsheets and paper, automates compliance alerts, and provides real-time visibility to managers and employees.

Key Features

1. **Smart Scheduling**
 - Drag-and-drop calendar for shifts.
 - An easy way for employees to put in PTO request.
 - An easy way for other employees to see what shifts are available.
 - Way for employees to show they are available for the open shifts.
 - Conflict detection (prevents double-booking or uncovered shifts).
2. **Overtime Tracking**
 - Automatic logging of hours worked, tied to scheduled vs. actual shifts.
 - Alerts when employees are approaching overtime thresholds.
 - Exportable reports for payroll and finance teams.
3. **Certification & Compliance Management**
 - Digital record of certifications with expiration dates.
 - Automated reminders to employees and supervisors before certifications are going to expire.
 - Restricts assignment of uncertified staff to regulated roles.
4. **Dashboards & Reporting**
 - Real-time overview of staffing levels, overtime, and compliance status.
 - Customizable reports for leadership, HR, and compliance auditors.
5. **User Roles & Access Control**
 - **Admins/Managers:** Full scheduling, compliance, and overtime views.
 - **Employees:** View shifts, request time off, show availability for shifts, upload certifications.
 - **Auditors:** Limited access to compliance records and reports.

Benefits

- **Efficiency Gains:** Cuts administrative time spent reconciling spreadsheets.
- **Cost Control:** Prevents unnecessary overtime and ensures accurate payroll.
- **Risk Reduction:** Automated compliance tracking avoids lapsed certifications.
- **Employee Satisfaction:** Transparent scheduling reduces confusion and last-minute changes.
- **Scalability:** Designed to grow with workforce size and regulatory complexity.

Stakeholders

Primary Stakeholders

- **Managers/Schedulers:** Need real-time visibility into staffing, overtime, and certifications to prevent conflicts and ensure compliance.
- **Employees/Staff:** Need an easy way to request time off, view schedules, claim open shifts, and upload certifications.
- **Compliance Officers / HR:** Require accurate certification records, expiration alerts, and compliance reports for audits.

Secondary Stakeholders

- **Finance/Payroll:** Benefit from automated overtime tracking and clean reporting for accurate labor cost calculations.
- **Executive Leadership:** Gain insights into operational efficiency, compliance risk, and workforce utilization.
- **IT/Admin Team:** Responsible for maintaining the system, managing access controls, and ensuring data security.

Scope

In Scope (Phase 1 / MVP)

- Basic employee database (name, role, certifications, availability).
- Shift scheduling module with:
 - Drag-and-drop interface.
 - Conflict detection.
 - PTO request workflow.
- Overtime tracking tied to scheduled hours.
- Certification record management with expiry alerts.
- Role-based access (Admin, Employee).

Out of Scope (Future Phases)

- Payroll system integration.
- Advanced mobile app features (push notifications, biometric log-in).
- Multi-language support.

- Complex reporting dashboards beyond basic overtime and certification status.

Success Metrics

Operational Metrics

- **Reduction in scheduling conflicts:** At least 80% fewer double-booked or uncovered shifts within the first 30 days.
- **Time saved by managers:** Reduce administrative scheduling time by 50% compared to paper/spreadsheet processes.
- **Certification compliance:** 100% of certifications tracked in the system with automated reminders at least 30 days before expiry.

Adoption Metrics

- 90% of employees log in to view schedules or submit PTO requests in the first month.
- At least 75% of shift changes/requests processed through the app instead of paper/email by the end of pilot period.

Satisfaction Metrics

- Manager satisfaction score $\geq 8/10$ on efficiency and usability survey.
- Employee satisfaction score $\geq 7/10$ on ease of use and transparency.

Minimal Viable Artifact (Shippable in 2 Weeks)

To demonstrate value quickly, the MVP should focus on **solving the biggest current pain point: scheduling and compliance visibility.**

MVP Deliverables

- **Web-based scheduling dashboard:** Managers can create/edit shifts, see conflicts, and approve/reject PTO.
- **Employee portal (basic):** Employees can log in, request PTO, view schedules, and mark availability for open shifts.
- **Certification tracker:** Upload certification info (date issued, expiration date), with a simple reminder system.
- **Role-based access:** Admins manage schedules/certs; employees request/view; auditors can see compliance status.
- **Basic reporting:** Export overtime hours and certification status into CSV/PDF.

Technology Stack (example for class project)

- Backend: Django (Python) with SQLite (simple, easy to deploy).
- Frontend: Django templates (or lightweight React if time allows).
- Authentication: Django's built-in user management.
- Hosting: Local demo or Heroku/Render (fast to spin up).

System Sketch

Architecture (block) diagram — GitHub + Django app

```
graph TD
    %% Actors
    A[Staff / Managers] ---|Browser| B[Web UI (Django templates or small React front end)]
    A2[Training / Compliance Officer] ---|Browser| B

    %% Web tier
    B --> C[Django Backend (Views/URLs)]
    C --> D[Domain Apps\n• Scheduling\n• Overtime\n• Certifications\n• Admin]
    C --> E[REST/GraphQL API (Django REST Framework)]

    %% Data & infra
    E --> F[(PostgreSQL)]
    E --> G[Redis Cache]
    C --> H[Celery Workers\n(batch jobs, reminders)]
    H --> F
    H --> I[Email/SMS Gateway\n(Twilio/SMTP)]
    C --> J[File/Object Storage (S3/GCS)\nattachments, exports]
```

Architecture Diagram

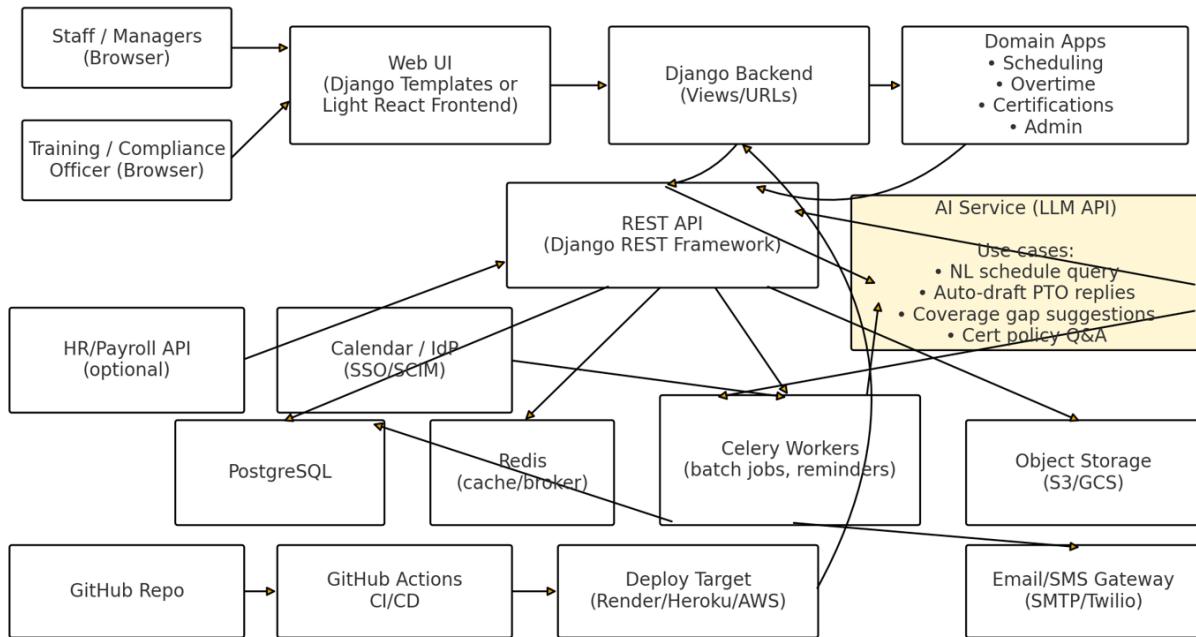
```
%% AI touchpoints
E -.-> K[[AI Service (LLM API)]]
H -.-> K
K -.-> E
K -.-> H
subgraph "AI Use Cases"
  K1[• Natural-language schedule query (NL→SQL/intents)]:::ai
  K2[• Auto-draft responses to time-off requests]:::ai
  K3[• Coverage gap detection & suggestion]:::ai
  K4[• Training/cert expiry explanation & policy Q&A]:::ai
end
K --- K1
K --- K2
K --- K3
K --- K4

%% External systems & CI/CD
E --> L[HR/Payroll API (optional)]
H --> M[Calendar/IdP (SSO/SCIM)]
N[GitHub Repo] -.PR/Merge.-> O[GitHub Actions CI/CD]
O --> P[Deploy target (e.g., Render/Heroku/AWS ECS)]
P --> C

classDef ai fill:#fff6d6,stroke:#f2b705,color:#111;
```


Legend (quick):

- AI touchpoints (yellow) = where the LLM is called to help (NL queries, auto-drafts, suggestions, policy Q&A).
- Celery handles scheduled jobs (daily certification sweeps, weekly overtime summaries, SMS/email reminders).
- Redis caches heavy reads and backs Celery as a broker.
- DRF API powers the web UI and any future mobile client.



Key user flow (sequence) manager submits a last-minute schedule change

Sequence Diagram

 Copy code

```
sequenceDiagram
```

```
    autonumber
```

```
    participant M as Manager (Browser)
```

```
    participant W as Web UI
```

```
    participant D as Django API (DRF)
```

```
    participant DB as PostgreSQL
```

```
    participant Q as Celery Worker
```

```
    participant AI as AI Service (LLM)
```

```
    participant MSG as Email/SMS Gateway
```

```
    M->>W: Open "Update Shift" form
```

```
    W->>D: POST /api/shifts/{id} (new assignee/time)
```

```
    D->>DB: Validate + write transaction
```

```
    DB-->>D: OK
```

```
    %% AI coverage check
```

```
    D->>AI: Analyze change → detect coverage gaps & suggest alternatives (skills, OT limits) <<AI t
```

```
    AI-->>D: Ranked suggestions + rationale
```

```
    %% Notify & assist
```

```
    D-->>W: 200 + suggestions (show inline "AI Suggestions" panel)
```

```
    M->>W: Accept suggestion (or keep original)
```

```
    W->>D: POST /api/shifts/{id}/confirm
```

```
    %% Commit + message ops
```

```
    D->>DB: Save final schedule
```

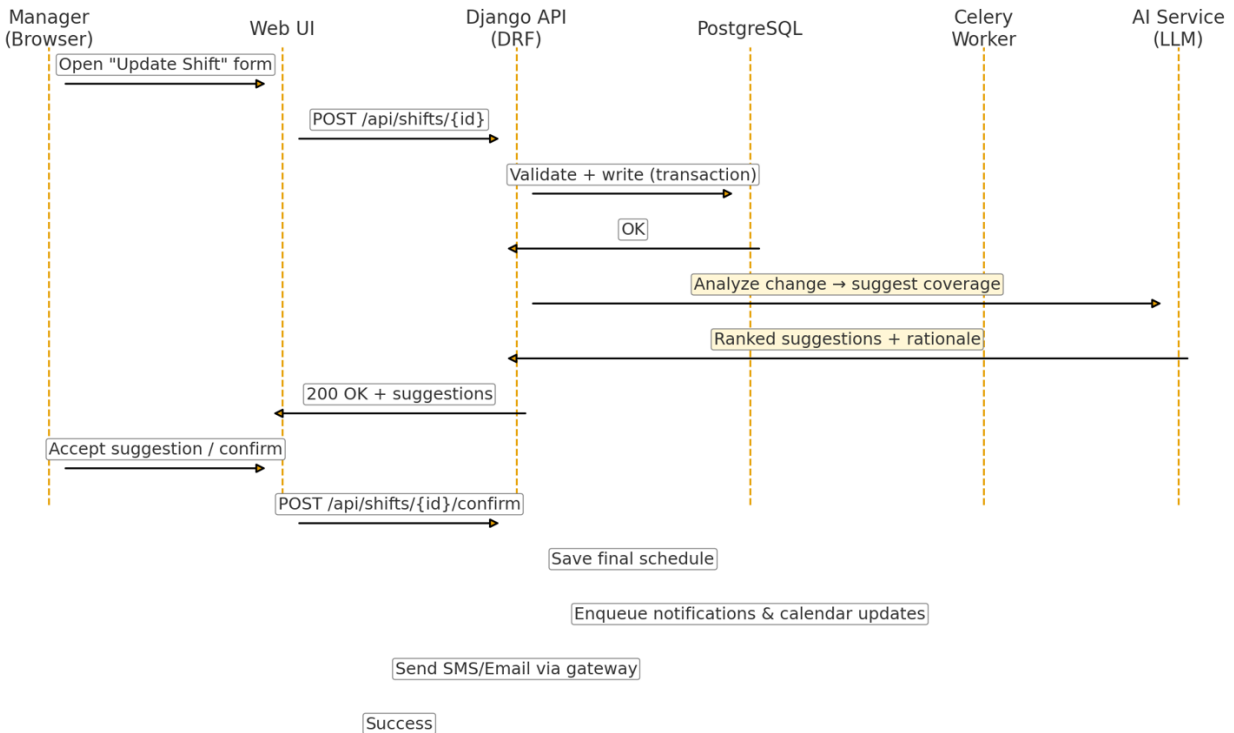
```
    D->>Q: Enqueue notifications & calendar updates
```

```
    Q->>MSG: Send confirmations to affected staff
```

```
    MSG-->>Q: Delivered
```

```
    Q-->>D: Done
```

```
    D-->>W: Success (shift updated, notices sent)
```



Optional AI touchpoints (Where and Why)

1. Natural-language schedule search: “Who’s certified for ERT and free after 18:00 on Friday?” (NL → intent → filtered query)
2. Coverage-gap detection: After any change, run a fast check (skills, certifications, max hours, OT policy, rest windows) and surface ranked suggestions.
3. Auto-draft responses: For denied/modified PTO, generate a respectful, policy-aligned explanation for manager review.
4. Certification assistant: Daily sweep flags upcoming expirations; AI generates personalized renewal steps and links to training.

Why this maps cleanly to GitHub + Django

- Django: fast CRUD for Scheduling/Overtime/Certifications + built-in Admin for back-office.
- DRF: clean API for web and (future) mobile.
- Celery + Redis: reliable reminders, daily compliance sweeps, digest emails.
- PostgreSQL: relational integrity for shifts↔people↔certs.
- GitHub Actions: lint/tests/migrations → deploy (e.g., Render/Heroku/AWS).
- LLM API: kept stateless; you pass only minimal, policy-safe context.

Ops & Delivery

Code lives in GitHub; GitHub Actions runs lint/tests/migrations and deploys to your target (Render/Heroku/AWS). Celery handles scheduled tasks, backed by Redis. Secrets are kept out of the repo and loaded via environment variables.

Evidence base: (3 authoritative sources)

1. According to Layman (2023), a modular Django architecture supports maintainability, testing, and secure deployment.
 - This source establishes best practices in Django’s structure—including modular apps, URL routing, forms, and CI/CD—that justify the architectural choices in our design.
2. Knight et al. (2023) found that AI-enhanced scheduling significantly reduces provider workload and improves outcomes.
 - This evidence demonstrates how AI and ML can optimize scheduling workflows, reduce burden, and increase satisfaction—validating the AI touchpoints in our system.
3. Del Gallo et al. (2023) showed that neural networks, reinforcement learning, and swarm optimization are effective in real-world scheduling.
 - This confirms that advanced AI methods are practical and effective for solving complex scheduling challenges, strengthening our approach to coverage-gap detection and automated suggestions.

Risk Register

Risk	Description	Impact	Likelihood	Mitigation Strategy
Data Privacy & Security	Sensitive employee scheduling and certification data could be exposed through misconfiguration, insecure APIs, or AI prompts	High	Medium	- Enforce role-based access control (RBAC) in Django. - Store secrets outside GitHub repo (use env vars). - Use HTTPS/TLS and Django security middleware. - Apply prompt sanitization when using

				AI APIs.
AI Reliability & Accuracy	AI-generated coverage suggestions or PTO responses may be incorrect, incomplete, or biased, leading to staffing gaps or employee dissatisfaction.	Medium-High	Medium	• Keep manager-in-the-loop for AI outputs. • Log and audit AI decisions. • Test against realistic edge cases. • Provide fallback manual workflows.
System Downtime or Deployment Errors	Outages during GitHub Actions deployment or Celery task failures could prevent schedule updates, notifications, or compliance sweeps.	High	Low-Medium	• Use staging environment for testing CI/CD pipelines. • Implement health checks and monitoring (e.g., Sentry, Prometheus). • Retry logic in Celery tasks. • Document manual failover process.

How I used AI on this project.

I wrote out my problem and solution and then asked the ChatGPT the following question.

Me: This is my problem, and proposed solution. I now need to define my stakeholders, scope, success metrics; define a minimal viable artifact shippable within 2 weeks.

AI: Gave me a basic outline with suggestions.

- I used the basic outline with suggestions and expanded on this to fit my needs.

Me: We will be using GitHub and Django to create this app. Based on that what would a
• System sketch: labeled diagram (block/sequence) showing front end back-end data
AI touchpoints look like

AI: Awesome—here's a clean, copy-paste sketch you can drop into your pitch. I've given you both a **block (architecture) diagram** and a **sequence diagram** with the AI touchpoints clearly labeled.

- ALL were copied and pasted into my document.

Me: I placed several resources for my evidence bases into ChatGTP and asked for help refining my Evidence base based on my readings.

AI: Gave me suggestions on how to integrate the sources into my report.

- I found this to be extremely helpful in pairing down all the things I was trying to use.

Me: can you give me suggestions for the top 3 risks with mitigations

AI: Gave me a basic risk register that said it aligned to my Django and GitHub and AI scheduling system

AI in GitHub

Me: This is my pitch and I need to create these deliverables in a github project using Django what would the steps be

AI: Gave me 6 steps to follow to create my first Repository

This is where I am having some trouble following.